Learn ESP32 for Home Automation

Lesson 6: Using MQTT for Home Automation

Using MQTT for Home Automation - Study Notes

Key Concepts

1. **Publish/Subscribe Model** – Devices (clients) publish messages to topics; other devices subscribe to those topics to receive updates.

2. **MQTT Broker** – Central server (e.g., Mosquitto, EMQX) that routes messages between publishers and subscribers.

3. **Topic Hierarchy** – Structured strings (e.g., `home/livingroom/light`) that organize data and simplify filtering.

4. **QoS Levels** – Quality of Service (0, 1, 2) determines delivery guarantees for each message.

5. **Retained Messages & Last Will** – Retained messages keep the latest state for new subscribers; Last Will notifies others if a client disconnects unexpectedly.

Summary

MQTT (Message Queuing Telemetry Transport) is a lightweight, publish/subscribe messaging protocol ideal for home automation scenarios where many low power devices need to exchange state information reliably. A central **MQTT broker** receives messages from **publishers** (e.g., a temperature sensor) and forwards them to any **subscribers** (e.g., a thermostat UI) that have expressed interest in the corresponding **topic**.

By organizing topics hierarchically (e.g., `home/kitchen/temperature`), you can easily control and monitor individual devices or whole groups. MQTT's three QoS levels let you balance network traffic against reliability, while retained messages ensure that a newly connected device instantly knows the current state of a light, door lock, or sensor. Using these features, a simple home automation system can be built where a smartphone app publishes a command like `home/livingroom/light/set` !' `ON`, and the light's firmware, subscribed to that topic, turns the lamp on and optionally publishes its new status back to `home/livingroom/light/state`.

Important Points

• **Broker selection:** Mosquitto is a popular open source broker; cloud options (AWS IoT, HiveMQ Cloud) add TLS and authentication out of the box.

• **Topic naming conventions:**

- Use lowercase, avoid spaces.
- Separate levels with `/`.
- Reserve the last level for the action (`set`, `state`, `config`).

• **QoS choices:**

- **0 – At most once:** fastest, no ack (good for frequent sensor data).

- **1 – At least once:** guarantees delivery, may duplicate (good for commands).
- **2 – Exactly once:** highest overhead, used sparingly.

• **Retained messages:** Set `retain=true` on a publish to store the last known state; useful for “status” topics.

• **Last Will and Testament (LWT):** Configure a client’s LWT so the broker publishes a predefined message if the client disconnects unexpectedly (e.g., `"offline"` status).

• **Security basics:**
- Enable TLS/SSL on the broker.
- Use username/password or client certificates.
- Restrict topic access with ACLs.

• **Payload format:** JSON is common (`{"state":"ON"}`) because it’s human readable and easy to parse on microcontrollers.

Examples

Example 1 – Turning a Light On/Off

Topic structure: `home/livingroom/light/set` (command) and `home/livingroom/light/state` (status)

Publisher (smartphone app):

```python
import paho.mqtt.publish as publish
publish.single(
    topic="home/livingroom/light/set",
    payload="ON",              # or "OFF"
    qos=1,
    retain=False,
    hostname="mqtt-broker.local"
)
```

Subscriber (ESP8266 light controller):

```python
import paho.mqtt.client as mqtt
import json, time
def on_message(client, userdata, msg):
    if msg.payload.decode() == "ON":
        turn_led(True)
    else:
        turn_led(False)
    # publish new state
    client.publish("home/livingroom/light/state", "ON" if msg.payload.decode()=="ON" else "OFF", retain=True)
client = mqtt.Client()
client.on_message = on_message
client.connect("mqtt-broker.local", 1883, 60)
```

```
client.subscribe("home/livingroom/light/set", qos=1)
client.loop_forever()
```

Explanation: The app publishes a command; the ESP receives it, actuates the LED, and then publishes the current state as a retained message so any future subscriber instantly knows the light's status.

Example 2 – Temperature Sensor with Retained State

Topic: `home/kitchen/temperature` (sensor publishes)

Sensor code (Arduino with PubSubClient):

```cpp
#include <PubSubClient.h>
WiFiClient espClient;
PubSubClient client(espClient);
void publishTemp(float t) {
  char payload[10];
  dtostrf(t, 1, 2, payload);
  client.publish("home/kitchen/temperature", payload, true); // retain = true
}
```

Dashboard subscriber (Node RED flow):

- MQTT input node subscribed to `home/+/temperature`.
- Passes payload to a gauge widget.

Explanation: Because the temperature message is retained, when the dashboard starts it immediately receives the latest reading without waiting for the next sensor cycle.

Quick Review

1. **What role does the MQTT broker play in a home automation network?**
2. **Name two advantages of using retained messages for device state topics.**
3. **When would you choose QoS /1 over QoS /0 for a command message?**
4. **Give an example of a well structured topic for a door lock's command and its status.**
5. **What is a Last Will message, and why is it useful for reliability?**

Further Reading

- **MQTT 5.0 features** – User properties, shared subscriptions, and reason codes.
- **Secure MQTT deployments** – TLS configuration, certificate management, and OAuth2 integration.
- **Home automation platforms** – Integrating MQTT with Home Assistant, OpenHAB, or Node RED.
- **Optimizing payloads** – Binary formats (CBOR, MessagePack) for low bandwidth devices.
- **Scalable broker architectures** – Clustering, load balancing, and high availability setups.